ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

СОГЛАСОВАНО	УТВЕРЖДЕНО
Приглашенный преподаватель	Академический руководитель образовательной программы "Программная инженерия", старший преподаватель департамента программной инженерии
_____ С. Б. Гурин	_____ Н. А. Павлочев
«___» _____ 2025 г.	«___» _____ 2025 г.

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подп	

SENTINEL — ИНТЕЛЛЕКТУАЛЬНЫЙ ТАЙМ-МЕНЕДЖЕР

Техническое задание

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.06.05-01 ТЗ 01-1-ЛУ

Исполнители:
Студент группы БПИ236
_____ / Т. С. Максимов /
«___» _____ 2025 г.

2025

УТВЕРЖДЕН

RU.17701729.06.05-01 ТЗ 01-1-ЛУ

SENTINEL — ИНТЕЛЛЕКТУАЛЬНЫЙ ТАЙМ-МЕНЕДЖЕР

Техническое задание

RU.17701729.06.05-01 ТЗ 01-1

Листов 30

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подп	

2025

АННОТАЦИЯ

Техническое задание – это основной документ, оговаривающий набор требований и порядок создания программного продукта, в соответствии с которым производится разработка программы ее тестирование и приемка.

Настоящее Техническое задание на разработку «Sentinel — Интеллектуальный тайм-менеджер» содержит следующие разделы: «Введение», «Основания для разработки», «Назначение разработки», «Требования к программе», «Требования к программной документации», «Технико-экономические показатели», «Стадии и этапы разработки», «Порядок контроля и приемки», приложения [7].

В разделе «Введение» указано наименование и краткая характеристика области применения программы.

В разделе «Основания для разработки» указан документ, на основании которого ведется разработка, и наименование темы разработки.

В разделе «Назначение разработки» указано функциональное и эксплуатационное назначение создаваемого программного продукта.

Раздел «Требования к программе» содержит указание на основные требования к функциональным характеристикам программы, к её надежности и к условиям эксплуатации, к составу и параметрам технических средств, к информационной и программной совместимости, к маркировке и упаковке, к транспортировке и хранению, а также специальные требования.

Раздел «Требования к программным документам» содержит указание на предварительный состав программной документации и специальные требования к ней.

Раздел «Технико-экономические показатели» содержит информацию об ориентировочной экономической эффективности разработки, экономические преимущества разработки программы.

Раздел «Стадии и этапы разработки» содержит информацию о стадиях разработки, этапах и содержании работ.

В разделе «Порядок контроля и приемки» указаны общие требования к приемке работы.

Настоящий документ разработан в соответствии с требованиями:

1. ГОСТ 19.101-77 [1]: Виды программ и программных документов.
2. ГОСТ 19.102-77 [2]: Стадии разработки.
3. ГОСТ 19.103-77 [3]: Обозначения программ и программных документов.
4. ГОСТ 19.104-78 [4]: Основные надписи.
5. ГОСТ 19.105-78 [5]: Общие требования к программным документам.

6. ГОСТ 19.106-78 [6]: Требования к программным документам, выполненным печатным способом.
7. ГОСТ 19.201-78 [7]: Техническое задание. Требования к содержанию и оформлению.

Изменения к данному Техническому заданию оформляются согласно ГОСТ 19.603-78 [12], ГОСТ 19.604-78 [13].

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ . . . 6
1.1. Наименование программы . . . 6
1.2. Краткая характеристика области применения программы . . . 6
2. ОСНОВАНИЯ ДЛЯ РАЗРАБОТКИ . . . 7
2.1. Документ(ы), на основании которого(ых) ведется разработка . . . 7
2.2. Наименование темы разработки . . . 7
3. НАЗНАЧЕНИЕ РАЗРАБОТКИ . . . 8
3.1. Функциональное назначение . . . 8
3.2. Эксплуатационное назначение . . . 8
4. ТРЕБОВАНИЯ К ПРОГРАММЕ . . . 9
4.1. Требования к функциональным характеристикам . . . 9
4.1.1. Требования к составу выполняемых функций . . . 9
4.1.2. Требования к организации входных данных . . . 9
4.1.3. Требования к организации выходных данных . . . 10
4.1.4. Требования к временным характеристикам . . . 10
4.1.5. Требования к интерфейсу . . . 10
4.1.6. Требования к хранению данных . . . 10
4.1.7. Интерфейсы API . . . 11
4.1.8. Формат файла для создания события (LLM → бэкенд) . . . 12
4.2. Требования к надежности . . . 13
4.3. Условия эксплуатации . . . 13
4.3.1. Климатические условия эксплуатации . . . 13
4.3.2. Требования к видам обслуживания . . . 14
4.3.3. Требования к численности и квалификации персонала . . . 14
4.4. Требования к составу и параметрам технических средств . . . 14
4.4.1. Программные средства . . . 14
4.4.2. Технические средства . . . 14
4.5. Требования к информационной и программной совместимости . . . 14
4.5.1. Требования к информационным структурам и методам решения . . . 14
4.5.2. Требования к программным средствам, используемым программой . . . 15
4.5.3. Требования к исходным кодам и языкам программирования . . . 15

4.5.4. Требования к защите информации и программы . . . 15
4.6. Требования к маркировке и упаковке . . . 15
4.7. Требования к транспортированию и хранению . . . 15
4.8. Специальные требования . . . 16
5. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ . . . 17
5.1. Состав программной документации . . . 17
5.2. Специальные требования к программной документации . . . 17
6. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ . . . 18
6.1. Предполагаемая потребность . . . 18
6.2. Целевая аудитория . . . 18
6.3. Преимущества перед аналогами . . . 18
6.3.1. Анализ аналогов . . . 18
6.3.1.1. Motion . . . 18
6.3.1.2. Reclaim.ai . . . 18
6.3.1.3. Sunsama . . . 19
6.3.1.4. Things 3 . . . 19
6.3.1.5. Todoist . . . 19
6.3.1.6. Notion Calendar . . . 19
6.3.1.7. Sentinel . . . 20
6.3.2. Дифференциаторы «Sentinel» . . . 20
6.3.3. Сравнительные критерии (матрица) . . . 20
7. СТАДИИ И ЭТАПЫ РАЗРАБОТКИ . . . 22
7.1. Стадии разработки, этапы и содержание работ . . . 22
7.2. Сроки разработки и исполнители . . . 24
8. ПОРЯДОК КОНТРОЛЯ И ПРИЕМКИ . . . 25
8.1. Виды испытаний . . . 25
8.2. Общие требования к приемке работы . . . 25
СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ . . . 26
ПРИЛОЖЕНИЕ А. ТЕРМИНОЛОГИЯ . . . 28
ПРИЛОЖЕНИЕ Б. ССЫЛКИ НА АНАЛОГИ . . . 29

1. ВВЕДЕНИЕ

1.1. Наименование программы

Наименование программы – «Sentinel — Интеллектуальный тайм-менеджер». Наименование программы на английском языке – «Sentinel — AI Scheduler».

1.2. Краткая характеристика области применения программы

Программа «Sentinel — Интеллектуальный тайм-менеджер» функционирует в клиент-серверной архитектуре, где iOS-приложение выступает клиентом, а серверная часть обеспечивает интерпретацию запросов и работу с данными.

Основной сценарий взаимодействия: пользователь в экране «Чат» формулирует запрос на естественном языке. Серверная часть интерпретирует запрос с использованием внешней языковой модели, выполняет поиск по персональным данным (события/напоминания) и возвращает структурированный результат, пригодный для создания или обновления записей календаря на стороне клиента.

Ключевые особенности серверной части:

1. Интеграция с внешним LLM для извлечения и нормализации сущностей (без собственных ML-моделей).

2. Единая модель «событие/напоминание» и операции поиска/создания/изменения/удаления через REST API.

3. Хранение истории чатов и сообщений для контекстного диалога.

4. Документированный API (OpenAPI 3.x), использование ISO-8601 и учёт часовых поясов (IANA).

5. Приватность и безопасность данных: строгое разграничение доступа к персональной информации.

2. ОСНОВАНИЯ ДЛЯ РАЗРАБОТКИ

2.1. Документ(ы), на основании которого(ых) ведется разработка

Разработка ведется на основании учебного плана подготовки бакалавров по направлению 09.03.04 «Программная инженерия» и утвержденной академическим руководителем программы темы курсового проекта.

2.2. Наименование темы разработки

Полное наименование темы: «Sentinel — Интеллектуальный тайм-менеджер (Клиентская часть)». Условное обозначение: «Sentinel».

3. НАЗНАЧЕНИЕ РАЗРАБОТКИ

3.1. Функциональное назначение

Назначение бэкенда — предоставление сервисов хранения и обработки данных, а также оркестрации диалогов с LLM для работы с персональным календарём пользователя.

Программа (серверная часть) должна обеспечивать выполнение следующих функций:

1. Обработка диалоговых запросов: интеграция с внешней языковой моделью для определения намерений пользователя и формирования структурированных запросов к данным.
2. Управление календарём: поиск («сегодня/завтра/на неделе» и по ключевым словам), создание, обновление и удаление событий и напоминаний (напоминание трактуется как событие без времени окончания).
3. Управление диалогами: ведение чат-сессий и сообщений, хранение текстовой истории взаимодействий.
4. Аутентификация и авторизация: регистрация/вход, выпуск и проверка токенов доступа, контроль прав на данные пользователя.
5. Совместимость и документация: предоставление REST API с описанием в OpenAPI 3.x, использование ISO-8601 для дат/времени и учёт часового пояса пользователя (IANA).
6. Достижения: выдача пользовательских достижений на основании заданных правил.

3.2. Эксплуатационное назначение

Бэкенд предназначен для эксплуатации как сетевой сервис, развернутый в контейнерах и доступный iOS-клиенту по защищённым REST-эндпоинтам. Требования к эксплуатации:

1. Наличие серверной инфраструктуры с современным программным обеспечением для веб-сервисов и реляционной СУБД.
2. Активное подключение к сети Интернет для вызовов внешней языковой модели.
3. Корректная обработка часовых поясов пользователя (IANA) при интерпретации формулировок «сегодня», «завтра», «на неделе».
4. Администрирование и сопровождение выполняются разработчиком/DevOps-специалистом в стандартных условиях эксплуатации серверного ПО.

4. ТРЕБОВАНИЯ К ПРОГРАММЕ

4.1. Требования к функциональным характеристикам

4.1.1. Требования к составу выполняемых функций

1. Обработка диалоговых запросов:

1.1. Интеграция с внешней языковой моделью для извлечения намерений и сущностей из текстовых запросов.

1.2. Определение намерений пользователя и формирование структурированных запросов к данным.

2. Управление календарём:

2.1. Поиск записей по временным диапазонам («сегодня», «завтра», «на неделе», произвольный интервал) и ключевым словам.

2.2. Создание, обновление и удаление событий и напоминаний (напоминание трактуется как событие без времени окончания).

3. Управление диалогами:

3.1. Ведение чат-сессий и сообщений, хранение текстовой истории взаимодействий.

3.2. Протоколирование вызовов инструментов (аргументы/результаты) при необходимости.

4. Аутентификация и авторизация:

4.1. Регистрация/вход, выдача и проверка токенов доступа (JWT), контроль прав на данные пользователя.

5. Совместимость и документация:

5.1. REST API с описанием в OpenAPI 3.x.

5.2. Использование ISO-8601 для дат/времени и учёт часового пояса пользователя (IANA).

6. Достижения:

6.1. Выдача пользовательских достижений на основании заданных правил.

4.1.2. Требования к организации входных данных

1. Формат обмена: JSON по HTTPS.

2. Аутентификация: заголовок Authorization: Bearer (для защищённых эндпоинтов).

3. Даты/время: ISO-8601 (UTC) с обязательным учётом пользовательской тайм-зоны (IANA) на уровне сервиса.

4. Диалоговые сообщения: текстовые запросы пользователя, структурированные данные инструмента (tool-args), идентификаторы чат-сессий.

5. События/напоминания: title (строка), start_at (ISO-8601), end_at (ISO-8601 или NULL для напоминаний), all_day (bool), type (event/reminder), location (опц.), tags (опц.).

6. Ограничения: валидация схемы входных данных, проверка прав доступа к ресурсам пользователя.

4.1.3. Требования к организации выходных данных

1. Формат ответов: JSON; коды ответов HTTP соответствуют семантике (2xx/4xx/5xx).

2. Ответы поиска событий: массив items[] с полями id, type (event/reminder), title, start_at, end_at, location (опц.), source; при необходимости формируется summary.

3. Ответы по чатам: хронологический список сообщений с ролями (user/assistant/tool/system) и техническими полями (опц.).

4. Идентификаторы: UUID (строка).

5. Ошибки: единый формат {code, message, details} с описанием причины и, при необходимости, полями неверной валидации.

4.1.4. Требования к временным характеристикам

1. Время ответа API для простых операций (чтение списков/деталей): ≤ 500 мс при номинальной нагрузке.

2. Поиск событий в БД (диапазон + текст): медиана ≤ 200–300 мс.

3. Синхронный диалоговый запрос с LLM: конечный ответ ≤ 8 с; при превышении — асинхронный сценарий/уточнение.

4. Вставка/обновление события в БД: ≤ 300 мс (без внешних вызовов).

4.1.5. Требования к интерфейсу

Версионирование REST: префикс /api/v1.

4.1.6. Требования к хранению данных

Система должна обеспечивать надёжное хранение следующих сущностей:

1. Пользователи: уникальная идентификация, контактная информация (email), учётные данные, настройки часового пояса (IANA) и локали, пользовательские предпочтения, временные метки регистрации и последнего входа.

2. События и напоминания: уникальный идентификатор, связь с пользователем, заголовок, описание, временные рамки начала и окончания, признак события на весь день, тип (событие/напоминание), местоположение, статус, источник создания (пользователь/ИИ/импорт), способ создания (чат/интерфейс/API), теги, временные метки создания и обновления.

3. Чаты: уникальный идентификатор, связь с пользователем, заголовок, назначение, временная метка последнего сообщения, признак архивации, временные метки создания и обновления.

4. Сообщения чатов: уникальный идентификатор, связь с чатом, роль отправителя (пользователь/ассистент/инструмент/система), текстовое содержимое, структурированное содержимое, данные вызова инструментов (имя, аргументы, результат), информация о провайдере ИИ и модели, временная метка создания.

5. Достижения пользователей: уникальный идентификатор, связь с пользователем, код достижения (уникальный в рамках пользователя и уровня), заголовок, описание, уровень, количество баллов, временная метка получения, дополнительные метаданные.

6. Требования к организации хранения:

6.1. Уникальная идентификация всех сущностей.

6.2. Поддержка связей между сущностями (пользователь-события, пользователь-чаты, чат-сообщения).

6.3. Корректная обработка часовых поясов с использованием стандарта IANA.

6.4. Поиск событий по временным диапазонам с медианным временем отклика не более 300 мс и полнотекстовый поиск по заголовкам и описаниям событий.

6.5. Поддержка хранения структурированных данных (настройки, метаданные, аргументы инструментов).

6.6. Обеспечение уникальности критичных полей (email пользователя, комбинация код-уровень достижения).

4.1.7. Интерфейсы API

1. Аутентификация:

1.1. POST /api/v1/auth/register — регистрация пользователя.

1.2. POST /api/v1/auth/login — вход, выдача JWT.

2. Диалоги/чаты:

2.1. GET /api/v1/chats — список чатов текущего пользователя.

2.2. POST /api/v1/chats — создать чат-сессию.

2.3. GET /api/v1/chats/{id}/messages — история сообщений.

2.4. POST /api/v1/chats/{id}/messages — отправить сообщение (сохранение в историю).

2.5. POST /api/llm/chat — оркестратор диалога (system prompt, tools, tool-use).

3. События и напоминания:

3.1. GET /api/v1/events?range=…&q=… — поиск событий по диапазону (today/tomorrow/this_week/next_week/custom) и строке запроса.

3.2. POST /api/v1/events — создать событие/напоминание.

3.3. PATCH /api/v1/events/{id} — обновить событие/напоминание.

3.4. DELETE /api/v1/events/{id} — удалить событие/напоминание.

4. Достижения:

4.1. GET /api/v1/achievements — achievements текущего пользователя.

Примечания:

1. Версионирование API — /api/v1; контракты описываются в OpenAPI 3.x.
2. Все защищённые ручки принимают JWT в заголовке Authorization: Bearer <token>.
3. Даты/время — ISO-8601; ответ поиска событий содержит items[] {id, type, title, start_at, end_at?, location?, source} и, при необходимости, summary.
4. Документация: OpenAPI 3.x; стабильность контрактов, обратная совместимость минорных изменений.
5. Единые соглашения по кодам ответов, пагинации, фильтрации и сортировке.
6. Локализация ответов ассистента — по умолчанию ru; системные поля/контракты — на английском.

4.1.8. Формат файла для создания события (LLM → бэкенд)

1. Требование: при подтверждении создания события диалоговой моделью формируется и передаётся структурированный JSON-файл с валидируемой схемой данных.

2. Основной формат: event_payload.json (Content-Type: application/json; charset=utf-8).

3. Способ передачи: как часть ответа инструмента (tool-use) в структурированном виде либо как текстовый блок, однозначно выделяемый оркестратором; постоянное хранение файлов не требуется — файл обрабатывается на лету и данные сохраняются в БД.

4. Обязательные поля JSON:

4.1. title: строка (не пустая).

4.2. start_at: строка в ISO-8601 (с тайм-зоной пользователя).

4.3. type: enum {event, reminder}.

4.4. all_day: boolean.

5. Дополнительные поля:

5.1. end_at: строка ISO-8601 (опционально; при наличии должно быть строго > start_at).

5.2. location: строка (опц.).

5.3. tags: массив строк (опц.).

5.4. source: строка, по умолчанию «llm». 5.5. created_via: строка, по умолчанию «chat».

6. Пример JSON (в одну строку):

6.1. {«title»:«Стрижка»,«start_at»:«2025-02-10T15:30:00+03:00»,«end_at»:«2025-02-10T16:00:00+ [«personal»],«source»:«llm»,«created_via»:«chat»}

7. Альтернативный формат (опционально): text/calendar (.ics). Поддержка ICS может быть добавлена позднее; приоритет — JSON.

8. Валидация: ISO-8601 для дат/времени; проверка, что end_at > start_at (если задано); соответствие enum-полей; нормализация тайм-зоны (IANA) на уровне сервиса.

4.2. Требования к надежности

1. Идемпотентность операций создания событий (поддержка X-Idempotency-Key на ограниченное окно времени).

2. Тайм-ауты и повторы с джиттером для внешних LLM-вызовов; отказоустойчивость за счёт graceful degradation (сообщение пользователю о временной недоступности).

3. Контроль целостности данных: транзакции БД для критичных операций; валидация на уровне схем.

4. Логи и базовый аудит ключевых действий (создание/изменение/удаление событий).

5. Целевое значение доступности сервиса — не ниже 99.5% (для учебного проекта — как ориентир).

4.3. Условия эксплуатации

4.3.1. Климатические условия эксплуатации

Применимы стандартные условия эксплуатации серверного программного обеспечения; климатические требования определяются условиями размещения серверной инфраструктуры и провайдера облака.

4.3.2. Требования к видам обслуживания

1. Регулярное обновление зависимостей и установленных пакетов.

2. Мониторинг логов и основных метрик производительности.

3. Резервное копирование БД согласно регламенту (ежедневно, с возможностью восстановления до точки во времени).

4.3.3. Требования к численности и квалификации персонала

1. Backend-разработчик с опытом разработки веб-сервисов и работы с реляционными СУБД.

2. При необходимости — DevOps-специалист для CI/CD и развёртывания.

3. Пользователь конечного приложения специальных навыков администрирования не требует.

4.4. Требования к составу и параметрам технических средств

4.4.1. Программные средства

1. Серверная платформа с поддержкой асинхронной обработки HTTP-запросов.

2. Веб-фреймворк с поддержкой REST API, автоматической валидации входных данных и генерации документации API.

3. Реляционная СУБД с поддержкой транзакций, корректной обработки часовых поясов и полнотекстового поиска с использованием индексов.

4. Библиотека для работы с реляционной СУБД с поддержкой асинхронных операций и управления миграциями схемы данных.

5. Клиентская библиотека для интеграции с внешними языковыми моделями с поддержкой вызова инструментов (function calling/tool use).

6. Средства контейнеризации для упрощения развёртывания и масштабирования сервиса.

4.4.2. Технические средства

1. Минимальные требования для окружения разработки и тестового развёртывания:

1.1. CPU: 2 vCPU, RAM: 4 ГБ, SSD: 20 ГБ, публичный доступ в Интернет.

2. Для продуктивного окружения параметры могут быть увеличены согласно нагрузке (например, 4 vCPU, 8 ГБ RAM).

4.5. Требования к информационной и программной совместимости

4.5.1. Требования к информационным структурам и методам решения

1. Обмен данными — JSON по HTTPS; кодировка UTF-8.

2. Структуры домена:

2.1. Событие/напоминание: {id, type, title, start_at, end_at, all_day, location?, status, tags?}.

2.2. Сообщение чата: {id, role, content_text, created_at, tool_name?, tool_args?, tool_result?}.

3. Методы решения: поиск событий по диапазону дат (с учётом часовых поясов) и текстовым критериям; нормализация свободных формулировок («на неделе», «завтра») в точные временные интервалы ISO-8601.

4.5.2. Требования к программным средствам, используемым программой

1. Веб-сервер с поддержкой асинхронной обработки запросов и протокола HTTPS.

2. Реляционная СУБД с поддержкой расширений для ускорения текстового поиска.

3. Библиотека для работы с СУБД с поддержкой асинхронных операций и автоматического управления схемой данных.

4. Клиент для интеграции с внешней языковой моделью с обязательной обработкой сетевых тайм-аутов и ошибок.

4.5.3. Требования к исходным кодам и языкам программирования

1. Использование современного языка программирования с развитой экосистемой библиотек для веб-разработки.

2. Исходный код должен соответствовать принятым стандартам качества, поддерживать статическую типизацию и быть документированным.

3. Структура кода должна быть организована по слоям (API, бизнес-логика, доступ к данным, модели) для обеспечения модульности и тестируемости.

4.5.4. Требования к защите информации и программы

1. Аутентификация по JWT, хранение паролей только в виде безопасных хешей.

2. Секреты и ключи — в переменных окружения/secret store.

3. Доступ к персональным данным — строго в пределах владельца; все запросы авторизованы.

4. Передача данных — только по HTTPS; защита от базовых атак (rate-limit, валидация входных данных).

4.6. Требования к маркировке и упаковке

1. Поставка серверной части в виде контейнерного образа с семантической версией.

2. Сопроводительная документация: README (развёртывание/переменные окружения), OpenAPI спецификация.

4.7. Требования к транспортированию и хранению

1. Хранение исходных кодов — в системе контроля версий.

2. Хранение образов — в реестре контейнеров.

3. Резервное копирование БД — по регламенту; хранение дампов в защищённом хранилище.

4.8. Специальные требования

1. Версионирование API (/api/v1), сохранение обратной совместимости минорных релизов.

2. Локализация ответов ассистента — русский язык; учёт часовых поясов (IANA).

3. Ограничения учебного проекта: отсутствие офлайн-вычислений LLM; при недоступности провайдера — информирование пользователя.

5. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

5.1. Состав программной документации

1. «Sentinel — Интеллектуальный тайм-менеджер». Техническое задание (ГОСТ 19.201-78 [7]).
2. «Sentinel — Интеллектуальный тайм-менеджер». Пояснительная записка (ГОСТ 19.404-79 [10]).
3. «Sentinel — Интеллектуальный тайм-менеджер». Программа и методика испытаний (ГОСТ 19.301-79 [8]).
4. «Sentinel — Интеллектуальный тайм-менеджер». Текст программы (ГОСТ 19.401-78 [9]).
5. «Sentinel — Интеллектуальный тайм-менеджер». Руководство оператора (ГОСТ 19.505-79 [11]).

5.2. Специальные требования к программной документации

Документы к программе должны быть выполнены в соответствии с ГОСТ 19.106-78 и ГОСТами к каждому виду документа (см. п. 5.1.); Пояснительная записка должна быть загружена в систему Антиплагиат через LMS «НИУ ВШЭ».

Техническое задание и пояснительная записка, титульные листы других документов должны быть подписаны руководителем разработки и исполнителем.

Документация и программа сдается в электронном виде в формате .pdf или .docx. в архиве формата .zip или .rar;

За две недели до защиты комиссии все материалы курсового проекта: программная документация, программный проект, исполняемый файл, отзыв руководителя, отчет системы Антиплагиат, должны быть загружены одним или несколькими архивами в проект дисциплины «Курсовой проект» в личном кабинете в информационной образовательной среде SmartLMS НИУ ВШЭ.

6. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

6.1. Предполагаемая потребность

Программа удовлетворяет потребность пользователей в автоматизированной конвертации текстовых запросов на естественном языке в структурированные события календаря. Использование технологий искусственного интеллекта позволяет автоматизировать извлечение атрибутов событий (дедлайны, приоритеты), сокращая время на планирование.

6.2. Целевая аудитория

Целевой аудиторией программы являются студенты и специалисты, активно использующие мобильные устройства Apple (iPhone, iPad) для планирования своего времени и нуждающиеся в инструментах повышения личной продуктивности.

6.3. Преимущества перед аналогами

Разрабатываемое приложение «Sentinel» решает задачу генерации календарных событий на основе текстового диалога с ассистентом и дополнительно учитывает индивидуальные ограничения пользователя через модель «Resource Battery», автоматическую балансировку «Rebalance» и систему достижений. Ниже приведён взвешенный анализ рынка.

6.3.1. Анализ аналогов

6.3.1.1. Motion

Плюсы:

1. Автопланирование задач и пересборка расписания по приоритетам.
2. Интеграция с основными календарями и таск-системами.
3. Инструменты для командной работы и слотов для встреч.

Минусы:

1. Высокая стоимость подписки.
2. Непрозрачное поведение автопланировщика, который может менять план без явного контроля.
3. Достаточно высокий порог входа по UX и настройке.

6.3.1.2. Reclaim.ai

Плюсы:

1. Автоматический тайм-блокинг задач, привычек и фокуса.
2. Глубокая интеграция с Google Calendar, Slack и популярными таск-системами.
3. Календарный sync между несколькими календарями и защита личных слотов.
4. Аналитика и командные сценарии.

Минусы:

1. Жёсткая привязка к Google/Outlook, без прямой поддержки Apple Calendar.
2. Нет нативных приложений, фокус на веб/PWA.
3. Требует аккуратной первичной настройки рабочих часов и правил.

6.3.1.3. Sunsama

Плюсы:

1. Ежедневные ритуалы планирования и рефлексии.
2. Спокойная визуализация дня, сочетание задач и календаря.
3. Интеграции с популярными менеджерами задач.
4. Кроссплатформенность (веб + десктоп + мобильные приложения).

Минусы:

1. Отсутствие AI-автопланирования, упор на ручной контроль.
2. Высокая цена и отсутствие постоянного бесплатного тарифа.
3. Заметный объём ручных действий при плотном графике.

6.3.1.4. Things 3

Плюсы:

1. Нативный, быстрый и стабильный клиент в экосистеме Apple.
2. Удобная модель организации задач (Projects, Areas, Today, теги).
3. Надёжная офлайн-работа и простая синхронизация между устройствами Apple.

Минусы:

1. Почти нет прямых интеграций с внешними сервисами.
2. Нет продвинутой автоматизации и AI-логики, всё строится на ручных сценариях.

6.3.1.5. Todoist

Плюсы:

1. Мощные фильтры, метки и естественный ввод дат.
2. Кроссплатформенность и большая экосистема клиентов и расширений.
3. Интеграции с множеством внешних сервисов и автоматизаций.
4. Встроенная геймификация через систему Karma.

Минусы:

1. Продвинутые функции (фильтры, напоминания и др.) в платных тарифах.
2. При большом количестве проектов и меток интерфейс быстро захламляется.
3. Зависимость от облачной синхронизации.

6.3.1.6. Notion Calendar

Плюсы:

1. Глубокая связка с Notion и базами данных.
2. Совместная работа, шаринг и scheduling-линки.
3. Бесплатная часть экосистемы Notion.

Минусы:

1. Поддержка только Google Calendar, без нативного Apple/Outlook.
2. Относительно базовый функционал календаря по сравнению с профильными AI-решениями.
3. Просадки по производительности и удобству на крупных воркспейсах.

6.3.1.7. Sentinel

Плюсы:

1. Генерация нескольких событий из одного чат-запроса с предварительным просмотром.
2. Учёт «энергии дня» (Resource Battery) при подсказках и планировании.
3. Прозрачная ручная ребалансировка расписания.
4. Нативная интеграция с Apple Calendar через EventKit без сторонних облачных прослоек.
5. Экран достижений с геймификацией, завязанной на режим и энергию.

Минусы:

1. Отсутствие кроссплатформенности вне экосистемы Apple.
2. Нет глубоких интеграций с корпоративными таск-системами уровня Jira/Asana.

6.3.2. Дифференциаторы «Sentinel»

1. Генерация нескольких событий из одного чат-запроса с отметкой для добавления.
2. Учёт «энергии дня» (Resource Battery) при подсказках и планировании.
3. Автоматическая балансировка «Rebalance» для упорядочивания.
4. Нативная интеграция с Apple Calendar (EventKit) без сторонних облачных прослоек.
5. Система достижений с отдельным экраном «Достижения».

Расчёт экономической эффективности не производится в рамках курсового проекта.

6.3.3. Сравнительные критерии (матрица)

Критерий	Todoist	Google Calendar	Motion	Reclaim.ai	Sunsama	Things 3	Notion Calendar	Sentinel
Отображение событий	+	+	+	+	+	+	+	+
Генерация через ИИ	-	-	-	+	-	-	-	+
Интеграция с Apple Calendar	-	+	-	-	-	+	-	+
Ребалансировка расписания	-	-	+	+	-	-	-	+

Критерий	Todoist	Google Calendar	Motion	Reclaim.ai	Sunsama	Things 3	Notion Calendar	Sentinel
Модель энергии дня	-	-	-	-	-	-	-	+
Достижения/ геймификация	-	-	-	-	+	-	-	+
Кроссплатф.	+	+	+	+	+	-	+	-
Уведомления	+	+	+	+	+	+	+	+

Таблица 1. Сравнение функциональных характеристик

7. СТАДИИ И ЭТАПЫ РАЗРАБОТКИ

7.1. Стадии разработки, этапы и содержание работ

Стадии и этапы разработки были выявлены с учётом ГОСТ 19.102-77 [2].

Стадия разработки	Этап работ	Содержание работ	Исполнители работ	Сроки выполнения
Техническое задание	Обоснование необходимости разработки	Постановка задачи	Максимов Т. С.	
		Сбор исходных теоретических материалов	Максимов Т. С.	
	Научно-исследовательский этап разработки	Определение структуры входных и выходных данных	Максимов Т. С.	
		Предварительный выбор методов решения задач	Максимов Т. С.	
		Определение требований к техническим и программным средствам	Максимов Т. С.	
		Обоснование возможности решения поставленной задачи	Максимов Т. С.	
	Разработка и утверждение технического задания	Определение требований к программному продукту	Максимов Т. С.	
		Выбор языков программирования	Максимов Т. С.	
		Разработка и согласование технического задания с научным руководителем	Максимов Т. С.	
		Загрузка согласованного технического задания в SmartLMS	Максимов Т. С.	

Таблица 2. Стадии и этапы разработки

Стадия разработки	Этап работ	Содержание работ	Исполнители работ	Сроки выполнения
Рабочий проект	Разработка программы	Предварительная разработка структуры программы	Максимов Т. С.	
		Программирование и отладка программы	Максимов Т. С.	
	Разработка программной документации	Разработка документов в соответствии с требованиями ГОСТ 19 ЕСПД (Единой системы программной документации)	Максимов Т. С.	
	Испытания программы	Разработка, согласование и утверждение порядка в методики испытаний	Максимов Т. С.	
		Проведение испытаний программы в соответствии с утвержденными порядком и методикой	Максимов Т. С.	
		Корректировка программы и программной документации по результатам испытаний	Максимов Т. С.	
Разработка программы	Подготовка и передача программы	Подготовка программы и программной документации для презентации и защиты	Максимов Т. С.	

Стадия разработки	Этап работ	Содержание работ	Исполнители работ	Сроки выполнения
		Представление разработанного программного продукта научному руководителю и получение отзыва	Максимов Т. С.	

Стадия разработки	Этап работ	Содержание работ	Исполнители работ	Сроки выполнения
		Загрузка Пояснительной записки в систему Антиплагиат через ЛМС НИУ ВШЭ	Максимов Т. С.	
		Загрузка материалов курсового проекта в ЛМС, дисциплина «Курсовой проект, 3 курс, ПИ»	Максимов Т. С.	
		Защита программного продукта комиссии	Максимов Т. С.	

7.2. Сроки разработки и исполнители

Разработка должна быть завершена не позднее утвержденного срока сдачи курсового проекта. Исполнители: студенты группы БПИ236 Петянов Т.А. (Frontend/iOS) и Максимов Т.С. (Backend/ Python).

8. ПОРЯДОК КОНТРОЛЯ И ПРИЕМКИ

Контроль и приемка разработки осуществляются в соответствии с документом «Программа и методика испытаний» (ГОСТ 19.301–79 [8]).

8.1. Виды испытаний

Для проверки соответствия программы требованиям проводятся следующие виды испытаний:

Функциональные — чат, создание событий из ответов, работа с календарём.

Надёжность — устойчивость к сетевым сбоям и некорректным данным.

Производительность — время отклика интерфейса и обработки данных.

Удобство использования и доступность.

8.2. Общие требования к приемке работы

Работа считается выполненной при соблюдении следующих условий:

1. Успешная обработка не менее 90% текстовых запросов, фотографий и документов из тестового набора.
2. Отсутствие дублирования событий при экспорте в календарь.
3. Соблюдение временных ограничений на выполнение операций.
4. Отсутствие критических ошибок (Crash-free rate ≥ 99,5%).
5. Предоставление полного комплекта программной документации.

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. ГОСТ 19.101-77: Виды программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102-77: Стадии разработки. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103-77: Обозначения программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104-78: Основные надписи. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105-78: Общие требования к программным документам. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106-78: Требования к программным документам, выполненным печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.201-78: Техническое задание. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.301-79: Программа и методика испытаний. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.401-78: Текст программы. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.404-79: Пояснительная записка. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
11. ГОСТ 19.505-79: Руководство оператора. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
12. ГОСТ 19.603-78: Общие правила внесения изменений. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
13. ГОСТ 19.604-78: Правила внесения изменений в программные документы, выполненные печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
14. FastAPI Documentation. [Электронный ресурс], URL: https://fastapi.tiangolo.com/ (дата обращения: 29.11.2025).
15. Pydantic v2 Documentation. [Электронный ресурс], URL: https://docs.pydantic.dev/latest/ (дата обращения: 29.11.2025).

16. SQLAlchemy 2.0 Documentation. [Электронный ресурс], URL: https://docs.sqlalchemy.org/en/20/ (дата обращения: 29.11.2025).
17. Alembic Documentation. [Электронный ресурс], URL: https://alembic.sqlalchemy.org/en/latest/ (дата обращения: 29.11.2025).
18. asyncpg Documentation. [Электронный ресурс], URL: https://magicstack.github.io/asyncpg/current/ (дата обращения: 29.11.2025).
19. PostgreSQL Official Documentation. [Электронный ресурс], URL: https://www.postgresql.org/docs/ (дата обращения: 29.11.2025).
20. OpenAPI Specification 3.1. [Электронный ресурс], URL: https://spec.openapis.org/oas/latest.html (дата обращения: 29.11.2025).
21. OpenAI API Reference. [Электронный ресурс], URL: https://platform.openai.com/docs/api-reference (дата обращения: 29.11.2025).
22. OpenAI Function Calling / Tools. [Электронный ресурс], URL: https://platform.openai.com/docs/guides/function-calling (дата обращения: 29.11.2025).
23. RFC 7519: JSON Web Token (JWT). [Электронный ресурс], URL: https://www.rfc-editor.org/rfc/rfc7519 (дата обращения: 29.11.2025).
24. RFC 3339: Date and Time on the Internet: Timestamps. [Электронный ресурс], URL: https://www.rfc-editor.org/rfc/rfc3339 (дата обращения: 29.11.2025).
25. IANA Time Zone Database. [Электронный ресурс], URL: https://www.iana.org/time-zones (дата обращения: 29.11.2025).
26. JSON (RFC 8259) The JavaScript Object Notation (JSON) Data Interchange Format. [Электронный ресурс], URL: https://www.rfc-editor.org/rfc/rfc8259 (дата обращения: 29.11.2025).
27. Docker Documentation. [Электронный ресурс], URL: https://docs.docker.com/ (дата обращения: 29.11.2025).

ПРИЛОЖЕНИЕ А. ТЕРМИНОЛОГИЯ

1. **Event — Событие** — запись календаря с полями: заголовок, начало, окончание, заметки, идентификатор календаря.
2. **Resource Battery** — модель оценки текущего уровня энергии пользователя, учитывающая тип нагрузки и количество переключений контекста.
3. **Rebalance** — алгоритм автоматической оптимизации расписания путём перестановки событий.
4. **Chat — Чат** — диалог с ассистентом для получения предложений по событиям и действиям.
5. **EventKit** — системный фреймворк iOS для работы с календарями и напоминаниями.
6. **Apple Calendar** — системное приложение календаря в экосистеме Apple, используемое для хранения и отображения событий.
7. **Notion Calendar** — календарный продукт компании Notion, упомянутый в анализе аналогов.
8. **LLM (Large Language Model)** — большая языковая модель, используемая для интерпретации пользовательских запросов и извлечения сущностей.
9. **OpenAI SDK** — клиентская библиотека для обращения к внешним LLM-провайдерам из серверной части.
10. **JWT (JSON Web Token)** — формат токенов доступа для аутентификации и авторизации в API.
11. **ISO-8601** — международный стандарт представления даты и времени, применяемый в запросах и ответах API.
12. **IANA Time Zone Database** — база данных часовых поясов, используемая для корректного учёта пользовательской тайм-зоны.

ПРИЛОЖЕНИЕ Б. ССЫЛКИ НА АНАЛОГИ

Название	Ссылка
Motion	https://www.usemotion.com/
Reclaim.ai	https://reclaim.ai/
Sunsama	https://sunsama.com/
Things 3	https://culturedcode.com/things/
Todoist	https://todoist.com/
Notion Calendar	https://www.notion.so/product/calendar
Google Calendar	https://workspace.google.com/products/calendar/

Дата обращения: 29.11.2025.

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

Лист регистрации изменений									
Номера листов (страниц)					Всего листов (страниц в докум.)	№ документа	Входящий № сопроводительного докум. и дата	Подп.	Дата
Изм.	Измененных	Замененных	Новых	Аннулированных					